

Name: Andy Zhou **Period:** 7 **APCS Final Project Document**

Group Name: Acela Transit Authority, S.C.

Project Title: Celery City Transit Simulator

Project Description:

This is a simulation of a heavy-rail transit network in my city-building Minecraft server, inspired by the designs and aesthetics of online-game *Mini Metro* and various real-world transit diagrams and maps.

Functionalities:

1. Simulated automated train movement using linear interpolation (`lerp()`) across a predefined geometry of `ArrayList<Station>` objects
2. Continuous, multi-frequency dispatching using the “%” remainder operation on Processing’s `frameCount`.
3. Frame-based dwell times at designated station platforms mimic real-life transit networks.
4. Bidirectional terminus routing, mimicking real life (when a train reaches a terminal, it reverses direction towards the other terminal as opposed to stopping forever or despawning).
5. Live passenger flow using randomized generation math (simulated); all passengers disembark at terminal
6. Map featuring lines drawn at vertical, horizontal, and diagonal angles. Utilizes invisible waypoint nodes (“isHidden” stations) to ensure parallel track routing.

No external libraries are needed or used.

How does it work?

This simulation is fully automated and does not require any active input. It serves as a passive dashboard for those interested in understanding how modern transit networks look from a central command room.

To Run: Simply open the PDE file in Processing and press the play button.

To Expect: No key inputs or any other inputs are needed or included. The initial fleet of trains will immediately spawn from each line and begin navigating the colored transit lines. Every 3 seconds, a central dispatcher will deploy additional trains until the network reaches its global cap (3 per line). The user can watch the passenger counts increment next to station names (inside a parenthesis) and watch the trains “board” (simulated) those passengers that were waiting as each train pauses at a station node.

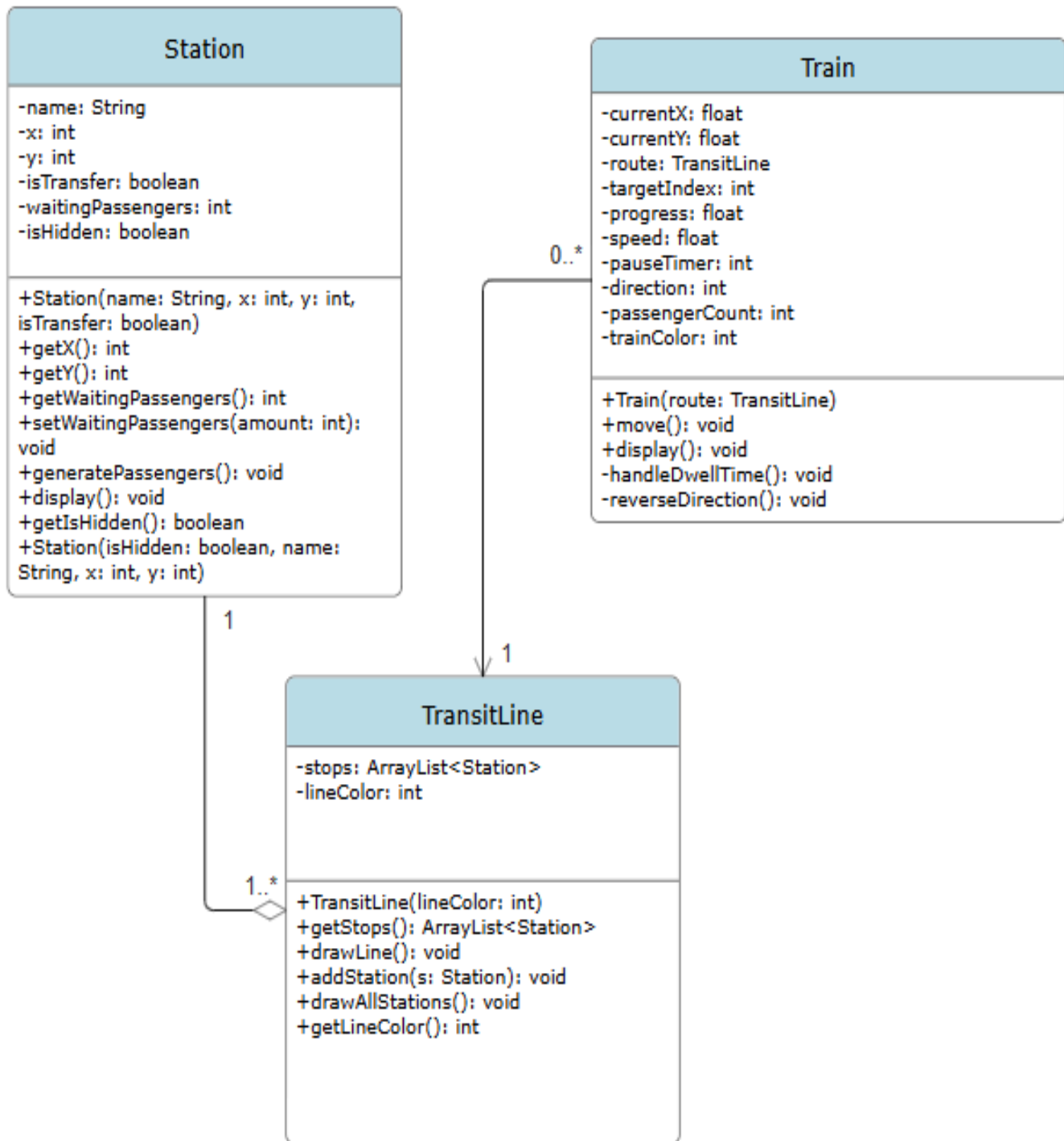
UML Diagram & Project Log to follow...

Name: Andy Zhou Period: 7 APCS Final Project Document

Group Name: Acela Transit Authority, S.C.

Project Title: Celery City Transit Simulator

UML Diagram



Project Log on the next page...

Name: Andy Zhou **Period:** 7 **APCS Final Project Document**

Group Name: Acela Transit Authority, S.C.

Project Title: Celery City Transit Simulator

Project Log (Working Alone; You (Andy) Zhou):

Section A: Core-Architecture

- First Commit: Created the GitHub repository and initial branch structure under core-architecture.
- 5/19 - Initiated work on Main class, Station, Train, and TransitLine.
- 5/19 - Finished initial setups of all methods; began working on the Station class; brainstormed (via comments) possible ways to continue this project as well as possible extension variables like boolean isExpress.
- 5/26 - Implemented arraylist traversal logic within TransitLine to render connected track geometry. Finished preparational setup in preparation for move() in all classes.
- 5/26 - Implemented various stations with accurate (read: approximate) coordinates based on real-world locations on my Minecraft server.
- 5/27 - Updated station class with new instance variable: name. Allows display of station names for all stations; realized oversight after running code drawing selected stations.
- 5/28 - Implemented the first version of lerp() in the Train method that does “work” but has a lot of bugs needing debugging.
- 5/28 - Continued implementing subway network stops. Added displayable station names to stops.
- 5/28 - Unified interchange stations; created a new line: Green line, with respective stop assignments
- 5/28 - Test trains added to all lines following initial success
- 5/28 - Added ability for trains to briefly dwell at “stations” using new instance variable
- 5/28 - Added commented-out instance variables as planned for weekend part 2
- 5/28 - ATA Metrorail continuous frequency has been implemented; trains currently operate every 3 seconds from terminal
- 5/28 - Preliminary completion of the branch Core-Architecture and the beginning of a new branch to-be-created for Advanced-Features.
- 5/28 - Finalized UML diagram for Advanced-Features implementation

Section B: Advanced-Features

- First Commit: Merged core-architecture with main and created new branch structure under advanced-features
- 5/29 - Initiated new instance variables, methods, helper methods, and modified select constructors in preparation for implementation of advanced-features
- 5/31 - Added reversal logic at all terminals & added global variable to cap per-line train spawning to 3 trains
- 5/31 - Added custom per-line colors to all trains and made each train a circle for maximum fluidity
- 5/31 - Added isHidden stations as vertices so train lines look more similar to real-world transit diagrams
- 5/31 - Added simulated live passenger count tracking at stations.
- 5/31 - Push & merge with main
- 5/31 - Debugged waiting passengers logic
- **5/31 - Finalized Project**